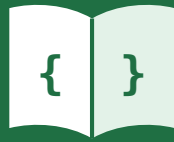

Modül 06 — Spring Data JPA ve Hibernate

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 JPA, Hibernate ve Spring Data JPA	2
2.2 Persistence Context ve Entity Yaşam Döngüsü	2
2.3 Şema Kimde?	3
3. Adım Adım Örnekler	3
3.1 Entity'ler ve İlişkiler	3
3.2 Türetilmiş Sorgular, Sayfalama ve Sıralama	5
3.3 N+1 Problemi	6
3.4 Projection'lar	7
3.5 Dinamik Arama: Specification	7
3.6 Auditing ve İyimser Kilitleme (@Version)	9
3.7 Kötümser Kilitleme	9
4. Sık Yapılan Hatalar ve En İyi Pratikler	10
5. Özet	11
6. İleri Okuma	11

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- JPA entity'lerini ve ilişkilerini (1-N, N-N) doğru biçimde eşlemek
- Spring Data JPA ile türetilmiş sorgular, sayfalama ve sıralama kullanmak
- N+1 sorgu problemini ölçerek tespit etmek ve @EntityGraph ya da join fetch ile çözmek
- Interface ve record projection'larıyla yalnızca gereken veriyi okumak
- Specification ile dinamik aramalar yazmak
- Denetim (auditing) alanlarını otomatik doldurmak
- İyimser (@Version) ve kötümser (SELECT ... FOR UPDATE) kilitlemeyi ayırt etmek

Ön koşullar: Modül 05 (Flyway, transaction'lar, Testcontainers) · **Tahmini süre:** 6 saat · **Docker gerekir**

2. Kavramlar

2.1 JPA, Hibernate ve Spring Data JPA

- **JPA (Jakarta Persistence):** Nesneleri tablolara eşlemenin standart API'sidir (@Entity, EntityManager).
- **Hibernate 7:** Spring Boot'un kullandığı JPA uygulamasıdır. SQL'i o üretir.
- **Spring Data JPA:** Repository arayüzlerinden sorgu üretir, sayfalama ve spesifikasyon desteği sağlar.

2.2 Persistence Context ve Entity Yaşam Döngüsü

Bir transaction içinde Hibernate, yüklediği her entity'yi bir **persistence context**'te (birinci seviye önbellek) tutar:

- Aynı id ikinci kez istenirse SQL çalıştırılmaz, aynı nesne döner.
- **Dirty checking:** Yönetilen bir entity'yi değiştirmek yeterlidir. Commit anında Hibernate farkı bulur ve UPDATE yazar. save() çağırma gerekmez.
- **Lazy loading:** İlişkiler (varsayılan olarak koleksiyonlar) ancak erişildiklerinde, açık bir transaction içinde yüklenir.

Durum	Anlamı
Transient	new ile yaratıldı, Hibernate bilmiyor
Managed	Persistence context içinde, değişiklikleri izleniyor

Durum	Anlamı
Detached	Transaction bitti, artık izlenmiyor
Removed	Silinmek üzere işaretlendi

2.3 Şema Kimde?

Bu kursta şemayı **Flyway** yönetir. Hibernate yalnızca entity'lerin şemayla uyumlu olduğunu doğrular (ddl-auto: validate). ddl-auto: update üretimde kullanılmamalıdır. Değişiklikleri izlenemez ve geri alınamaz.

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın:

```
./mvnw -pl modules/06-data-jpa-postgres/lesson -am spring-boot:run
```

JPA ayarları:

```
jpa:
  open-in-view: false           # no lazy loading in the web layer: queries stay in the
  ↪ service layer
  hibernate:
    ddl-auto: validate          # Flyway owns the schema; Hibernate only checks that
  ↪ entities match it
  properties:
    hibernate:
      generate_statistics: true # Lesson 3.3: count SQL statements (keep it off in
  ↪ production)
```

3.1 Entity'ler ve İlişkiler

Amaç: Yazar 1-N kitap ve kitap N-N kategori ilişkilerini eşlemek.

```
@Entity
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // bigserial in PostgreSQL
    private @Nullable Long id;
```

```

private String name;

@OneToMany(mappedBy = "author")           // Book.author owns the
    ↪ foreign key
private List<Book> books = new ArrayList<>(); // collections are LAZY by
    ↪ default

protected Author() {                     // required by JPA
    this.name = "";
}

```

```

@Entity
@EntityListeners(AuditingEntityListener.class) // fills @CreatedDate /
    ↪ @LastModifiedDate
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private @Nullable Long id;

    private String isbn;
    private String title;
    private BigDecimal price;
    private int stock;

    @ManyToOne(fetch = FetchType.LAZY, optional = false) // default for @ManyToOne is
        ↪ EAGER – avoid it
    @JoinColumn(name = "author_id")
    private Author author;

    @ManyToMany
    @JoinTable(name = "book_category",
        joinColumns = @JoinColumn(name = "book_id"),
        inverseJoinColumns = @JoinColumn(name = "category_id"))
    private Set<Category> categories = new HashSet<>();

    @CreatedDate
    private @Nullable Instant createdAt;

    @LastModifiedDate
    private @Nullable Instant updatedAt;

    @Version // optimistic locking
    private @Nullable Long version;
}

```

Entity'ler değişebilir sınıflardır, record değildir. Durumlarını setter'larla değil, anlamlı me-

totlarla değiştirirler:

```
public void changePrice(BigDecimal newPrice) {           // behaviour instead of setters
    if (newPrice.signum() < 0) {
        throw new IllegalArgumentException("Price must not be negative");
    }
    this.price = newPrice;
}

public void removeStock(int quantity) {
    if (quantity > stock) {
        throw new IllegalStateException("Only " + stock + " left");
    }
    this.stock -= quantity;                               // no save() needed: dirty
    ↪ checking at commit
}
```

Beklenen çıktı:

```
== 3.1 Entities and relationships
Effective Java by Joshua Bloch, categories [best-practices, java]
```

Testi: catalog/BookRepositoryTest.mapsTheRelationships

! Uyarı

@ManyToOne ve @OneToOne varsayılan olarak **EAGER**'dir. Her kitap okunduğunda yazar da okunur. Hep `fetch = FetchType.LAZY` yazın ve gerektiğinde bilinçli olarak birlikte yükleyin (bölüm 3.3).

3.2 Türetilmiş Sorgular, Sayfalama ve Sıralama

Amaç: Metot adından sorgu üretmek ve sonuçları sayfa sayfa almak.

```
Optional<Book> findByIsbn(String isbn);

Page<Book> findByTitleContainingIgnoreCase(String text, Pageable pageable); // paging +
↪ sorting for free
```

`PageRequest.of(0, 2, Sort.by("price").descending())` ile çağrıldığında Spring Data iki sorgu çalıştırır: sayfa verisi ve toplam sayı.

Beklenen çıktı:

```
== 3.2 Derived query with paging
[Modern Java in Action, Effective Java] – page 1 of 2, 3 matches
```

3.3 N+1 Problemi

Amaç: Performansın en sık görülen düşmanını ölçmek ve çözmek.

Tüm yazarları okuyup her birinin kitaplarına dokunmak $1 + N$ sorgu çalıştırır: 1 sorgu yazarlar için, sonra her yazar için ayrı bir sorgu. Çözüm, gereken ilişkiyi aynı sorguda yüklemektir:

```
public interface AuthorRepository extends JpaRepository<Author, Long> {

    Optional<Author> findByName(String name);

    // findAll() (inherited): 1 query for authors, then 1 query per author when books are
    ↪ touched → N+1

    @EntityGraph(attributePaths = "books") // fix 1: fetch the books in
    ↪ the same query
    List<Author> findAllWithBooksBy();

    @Query("select distinct a from Author a left join fetch a.books") // fix 2: JPQL
    ↪ join fetch
    List<Author> findAllFetchingBooks();
}
```

Test, Hibernate istatistikleriyle çalışan SQL sayısını ölçer:

```
@Test
void lazyCollectionsCauseOneQueryPerAuthor() {
    List<Author> all = authors.findAll();
    int books = countBooks(all);

    assertThat(books).isEqualTo(6);
    assertThat(statistics.getPrepareStatementCount()).isEqualTo(1 + all.size()); // 1 for
    ↪ authors + 1 per author
}
```

Beklenen çıktı:

```
== 3.3 N+1
findAll + touching books: 6 SQL statements
@EntityGraph:             1 SQL statement
join fetch:               1 SQL statement
```

★ **İpucu**

Geliştirirken `logging.level.org.hibernate.SQL: debug` ile üretilen SQL'i görün. Bir listeyi dolaşırken log'da aynı `select`'in tekrar tekrar görüldüğünü fark ederseniz, bu bir N+1'dir.

3.4 Projection'lar

Amaç: Entity'nin tamamını değil, yalnızca gereken sütunları okumak.

```
List<BookTitleAndPrice> findByAuthorNameOrderByTitle(String authorName);    //
↪ interface projection

@Query("""
    select new com.springbootedu.datajppostgres.catalog.BookCard(b.title, a.name,
↪ b.price)
    from Book b join b.author a
    where b.price < :max
    order by b.price""")
List<BookCard> findCardsCheaperThan(BigDecimal max);                        // record
↪ (DTO) projection
```

Projection'lar yönetilen entity değildir. Dirty checking ve lazy loading yoktur. Bu yüzden listeler ve raporlar için hem daha hızlı hem de daha güvenlidir.

Beklenen çıktı:

```
== 3.4 Projections
interface: Effective Java 89.90
interface: Java Puzzlers 55.00
record:    BookCard[title=Java Puzzlers, author=Joshua Bloch, price=55.00]
record:    BookCard[title=Refactoring, author=Martin Fowler, price=85.00]
```

3.5 Dinamik Arama: Specification

Amaç: Kullanıcının doldurduğu kadar kriterle, tek bir esnek sorgu kurmak.

Küçük, yeniden kullanılabilir sorgu parçaları:

```
final class BookSpecifications {

    private BookSpecifications() {
    }
}
```

```

static Specification<Book> titleContains(String text) {
    return (book, query, cb) -> cb.like(cb.lower(book.get("title")), "%" +
        ↪ text.toLowerCase(Locale.ROOT) + "%");
}

static Specification<Book> priceAtMost(BigDecimal max) {
    return (book, query, cb) -> cb.lessThanOrEqualTo(book.get("price"), max);
}

static Specification<Book> inCategory(String name) {
    return (book, query, cb) -> {
        Join<Book, Category> categories = book.join("categories");
        return cb.equal(categories.get("name"), name);
    };
}
}

```

Yalnızca dolu kriterler birleştirilir:

```

public Page<BookCard> search(BookFilter filter, Pageable pageable) {
    List<Specification<Book>> criteria = new ArrayList<>();
    if (filter.title() != null) {
        criteria.add(BookSpecifications.titleContains(filter.title()));
    }
    if (filter.maxPrice() != null) {
        criteria.add(BookSpecifications.priceAtMost(filter.maxPrice()));
    }
    if (filter.category() != null) {
        criteria.add(BookSpecifications.inCategory(filter.category()));
    }
    return books.findAll(Specification.allOf(criteria), pageable)
        .map(book -> new BookCard(book.getTitle(), book.getAuthor().getName(),
            ↪ book.getPrice()));
}

```

Beklenen çıktı (başlıkta “java”, en fazla 100, kategori “java”):

```

== 3.5 Specifications
Effective Java · Joshua Bloch · 89.90
Java Puzzlers · Joshua Bloch · 55.00

```

Testi: catalog/BookSearchTest

i Not

Alan adlarını metin olarak ("title") yazdık. Büyük projelerde hibernate-processor ile

üretilen tip güvenli metamodel (`Book_.title`) tercih edilir. Bir alanın adı değiştiğinde derleme hatası verir.

3.6 Auditing ve İyimsiz Kilitleme (@Version)

Amaç: Oluşturma ve güncelleme zamanlarını otomatik tutmak ve eşzamanlı güncellemelerde veri kaybını önlemek.

`@EnableJpaAuditing` (bkz. `JpaAuditingConfiguration`) ile `@CreateDate` ve `@LastModifiedDate` alanları kendiliğinden dolar. `@Version` alanı her güncellemede artar. Hibernate `UPDATE ... WHERE id = ? AND version = ?` yazar. Başka biri araya girip kaydı değiştirmişse satır bulunamaz ve `ObjectOptimisticLockingFailureException` fırlatılır.

Beklenen çıktı:

```
== 3.6 Auditing and @Version
price 86.00, version 1, updated 2026-09-23T15:50:27.597813Z
```

Testi: `catalog/AuditingAndVersioningTest`. İki kullanıcı aynı kitabı okur. İlki kaydeder, ikincisinin eskimiş kopyası reddedilir.

3.7 Kötümser Kilitleme

Amaç: Stok gibi “yarış” durumuna açık bir değeri, çok sayıda eşzamanlı alıcıyla güvenli azaltmak.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)           // SELECT ... FOR UPDATE
@Query("select b from Book b where b.isbn = :isbn")
Optional<Book> findForUpdate(String isbn);
```

```
@Service
public class StockService {

    private final BookRepository books;

    public StockService(BookRepository books) {
        this.books = books;
    }

    @Transactional
    public boolean trySell(String isbn, int quantity) {
        Book book = books.findForUpdate(isbn).orElseThrow();    // other buyers wait here
        ↪ until we commit
    }
}
```

```

        if (book.getStock() < quantity) {
            return false;
        }
        book.removeStock(quantity); // written at commit
    ↪ (dirty checking)
        return true;
    }
}

```

Beklenen çıktı (temiz bir veritabanıyla ilk çalıştırma):

== 3.7 Pessimistic locking: 10 buyers, 1 copy each, 3 in stock
stock before 3, sold 3, stock after 0

Testi: stock/StockServiceTest. 10 eşzamanlı alıcı ve 5 kopya varken tam 5 satış olur ve stok sıfıra iner.

	İyimser (@Version)	Kötümser (FOR UPDATE)
Nasıl?	Güncellemede sürüm karşılaştırılır	Satır okunurken kilitlenir
Çakışmada	Exception. İstemci yeniden dener	Diğerleri bekler
Uygun olduğu yer	Çakışmanın nadir olduğu durumlar (form düzenleme)	Sık çakışma, kısa işlemler (stok)

i Not

Kilit kaldırılırsa satış yine bozulmaz, çünkü @Version kayıp güncellemeleri yakalar. Ancak alıcıların yarısı bir exception alır. İki mekanizma birbirini tamamlar.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

spring.jpa.open-in-view varsayılan olarak **açıktır** ve lazy ilişkilerin web katmanında, veritabanı bağlantısı açık tutularak yüklenmesine izin verir. Bu, N+1'leri gizler ve bağlantı havuzunu tüketir. Bu kursta hep false yapıyoruz.

- **Yapın:** Tüm ilişkileri LAZY yapın. Gerektiğinde @EntityGraph veya join fetch ile birlikte yükleyin.

- **Yapmayın:** Entity'leri doğrudan JSON olarak döndürmeyin. Lazy ilişkiler ve sonsuz döngüler başınızı ağrıtır. DTO veya projection kullanın.
- **Yapın:** equals/hashCode'u doğal bir anahtara (ISBN) ya da dikkatle id'ye dayandırın. Kaydetmeden önce null olan id'ye dayanan bir hashCode koleksiyonları bozar.
- **Yapmayın:** İki yönlü ilişkilerde yalnızca bir tarafı güncellemeyin. addLine gibi yardımcı metotlarla iki tarafı birlikte kurun (Ödev 1).
- **Yapın:** Okuma yapan servislerde @Transactional(readOnly = true) kullanın. Hibernate dirty checking yapmaz.
- **Yapmayın:** IDENTITY id stratejisiyle toplu INSERT beklemeyin. Hibernate batch ekleme yapamaz. Çok sayıda kayıt için sequence veya JDBC batch (Modül 05) kullanın.

5. Özet

- Entity'ler değişebilir, persistence context tarafından yönetilen sınıflardır. Değişiklikler commit anında yazılır.
- Spring Data JPA, repository arayüzlerinden sorgu, sayfalama ve sıralama üretir.
- N+1'i ölçün. @EntityGraph veya join fetch ile tek sorguya indirin.
- Projection'lar okuma için hızlı ve güvenlidir. Specification dinamik sorgular kurar.
- Auditing zaman damgalarını, @Version eşzamanlı güncellemeleri yönetir.
- Yüksek çakışmalı değerler için kötümser kilit (PESSIMISTIC_WRITE) kullanılır.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Data JPA Reference](#)
- [Query Methods & Entity Graphs](#) · [Specifications](#) · [Projections](#)
- [Auditing](#) · [Locking](#)
- [Hibernate ORM 7 User Guide](#)
- [Spring Boot — SQL Databases \(JPA\)](#)